

Thực hành – Tuần 2C

Quản lý Bộ nhớ



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 26 tháng 3 năm 2026

Mục lục

I	Nội dung thực hành	2
I.1	Kiến thức	2
I.1.1	Vai trò của Quản lý bộ nhớ	2
I.1.2	Bộ nhớ tĩnh và Stack	4
I.1.3	Bộ nhớ động và Heap	8
I.1.4	Xử lý Phạm vi	12
I.2	Stack trong C++	16
I.3	Heap trong C++	17
I.4	Mảng trong C++	18
I.5	Ownership trong C++	19
I.6	Mục tiêu	19
II	Bài tập thực hành	21
III	Bài tập về nhà	29

I. Nội dung thực hành

I.1 Kiến thức

Trong phần lý thuyết trước đó, các khái niệm như *tên (name)*, *đối tượng (object)*, *liên kết (binding)*, *môi trường (environment)* và *phạm vi (scope)* đã được giới thiệu ở mức trừu tượng. Những khái niệm này giúp chúng ta mô tả cách một chương trình được hiểu và phân tích ở mức ngữ nghĩa.

Tuy nhiên, khi một chương trình thực sự được thực thi, các khái niệm trừu tượng đó phải được hiện thực hóa thông qua các cơ chế cụ thể của hệ thống runtime. Nói cách khác, chúng ta cần trả lời các câu hỏi như: một *đối tượng* được lưu trữ ở đâu trong bộ nhớ, *liên kết* giữa *tên* và *đối tượng* được duy trì bằng cách nào, và hệ thống thực thi làm thế nào để xác định *phạm vi* truy cập của các biến trong quá trình chạy chương trình.

Trong phần thực hành này, chúng ta sẽ khảo sát cách các khái niệm trên được hiện thực hóa trong ngôn ngữ C++. Cụ thể, chúng ta sẽ bắt đầu từ cơ chế *Quản lý bộ nhớ (Memory Management)*, từ đó xây dựng mô hình tổ chức bộ nhớ của chương trình và phân tích cách hệ thống runtime quản lý vòng đời cũng như phạm vi truy cập của các đối tượng dữ liệu.

I.1.1 Vai trò của Quản lý bộ nhớ

Vì sao cần quản lý bộ nhớ? Trong mọi ngôn ngữ lập trình, chương trình không chỉ bao gồm các chỉ thị thực thi mà còn bao gồm dữ liệu được tạo ra và biến đổi trong suốt quá trình chạy. Vì vậy, một trong những nhiệm vụ quan trọng của hệ thống thực thi là *Quản lý bộ nhớ (Memory Management)*.

Ở góc nhìn của *Máy trừu tượng (Abstract Machine)*, một chương trình được thực thi bởi một hệ thống runtime có nhiệm vụ mô phỏng hành vi của chương trình trên phần cứng thực tế. Hệ thống này phải đảm bảo rằng các đối tượng dữ liệu được cấp phát, truy cập và giải phóng một cách hợp lệ trong suốt quá trình thực thi.

Về bản chất, quản lý bộ nhớ phải giải quyết hai vấn đề cốt lõi đối với mọi đối tượng dữ liệu:

- *Vòng đời (Lifetime)* – đối tượng tồn tại trong khoảng thời gian nào của chương trình.
- *Khả năng truy cập (Accessibility)* – đối tượng có thể được tham chiếu từ những vị trí nào trong chương trình.

Một cơ chế quản lý bộ nhớ hiệu quả phải đảm bảo rằng mỗi đối tượng:

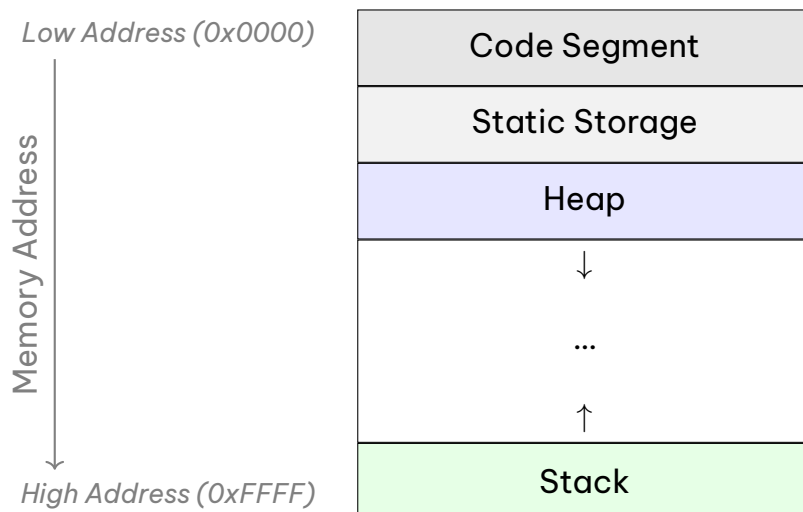
- được cấp phát khi cần thiết,
- tồn tại trong đúng khoảng thời gian cần thiết,
- và được giải phóng khi không còn sử dụng.

Nếu không có cơ chế quản lý bộ nhớ hợp lý, chương trình có thể gặp nhiều vấn đề nghiêm trọng như:

- truy cập vùng nhớ không hợp lệ,
- rò rỉ bộ nhớ (*memory leak*),
- hoặc ghi đè dữ liệu của các thành phần khác.

Mô hình bộ nhớ tổng quát của chương trình Để tổ chức dữ liệu trong quá trình thực thi, hầu hết các hệ thống runtime sử dụng một mô hình bộ nhớ tổng quát gồm bốn vùng chính:

- **Code Segment** - chứa mã máy của chương trình.
- **Static Storage** - chứa các biến toàn cục và các biến tĩnh.
- **Stack** - quản lý các lời gọi hàm và các biến cục bộ.
- **Heap** - vùng cấp phát bộ nhớ động trong runtime.



Hình 1: Mô hình bộ nhớ cơ bản của một chương trình

Mỗi vùng bộ nhớ được thiết kế để phục vụ cho một kiểu vòng đời khác nhau của dữ liệu trong chương trình. Trong một số hệ điều hành và kiến trúc khác, thì vị trí của **Stack** và **Heap** có thể đổi chỗ cho nhau.

Liên hệ với phạm vi của biến Trong lập trình, mỗi biến không chỉ có một vị trí trong bộ nhớ mà còn có một phạm vi truy cập xác định. Hai mô hình phổ biến là:

- *Phạm vi tĩnh (Static scope)* - phạm vi được xác định dựa trên cấu trúc tĩnh của chương trình.
- *Phạm vi động (Dynamic scope)* - phạm vi phụ thuộc vào chuỗi lời gọi hàm trong thời gian chạy.

Cách một ngôn ngữ lập trình xác định phạm vi của biến có mối liên hệ chặt chẽ với cách hệ thống runtime quản lý bộ nhớ. Nói cách khác, phạm vi truy cập của một biến thường gắn liền với vòng đời của đối tượng tương ứng trong bộ nhớ.

Trong các phần tiếp theo, chúng ta sẽ lần lượt nghiên cứu hai cơ chế quản lý bộ nhớ quan trọng:

- quản lý bộ nhớ theo **Stack** cho các đối tượng có vòng đời xác định rõ,
- và quản lý bộ nhớ theo **Heap** cho các đối tượng được tạo ra động trong runtime.

1.1.2 Bộ nhớ tĩnh và Stack

Bộ nhớ tĩnh Trong nhiều trường hợp, trình biên dịch có thể xác định trước kích thước và vị trí của các đối tượng dữ liệu ngay tại thời điểm biên dịch. Những đối tượng như vậy được quản lý theo cơ chế *cấp phát tĩnh (static allocation)*.

Các đối tượng được cấp phát tĩnh thường bao gồm:

- các biến toàn cục,
- các biến tĩnh trong hàm,
- và một số cấu trúc dữ liệu có kích thước cố định.

Những đối tượng này được lưu trữ trong vùng **Static Storage**. Vòng đời của chúng thường kéo dài trong toàn bộ thời gian thực thi của chương trình.

Tuy nhiên, nhiều đối tượng dữ liệu khác chỉ tồn tại trong thời gian thực thi của một lời gọi hàm. Những đối tượng này cần một cơ chế cấp phát linh hoạt hơn so với bộ nhớ tĩnh.

Cơ chế Stack Đối với các biến cục bộ và thông tin liên quan đến lời gọi hàm, hệ thống runtime thường sử dụng cấu trúc **Stack**.

Stack hoạt động theo nguyên lý *Last-In First-Out (LIFO)*. Điều này phù hợp với cấu trúc lời gọi hàm của chương trình: hàm được gọi sau cùng sẽ kết thúc trước.

Khi một hàm được gọi, hệ thống runtime sẽ tạo một vùng dữ liệu mới trên Stack để lưu trữ thông tin liên quan đến lời gọi hàm đó. Khi hàm kết thúc, vùng dữ liệu này sẽ được loại bỏ khỏi Stack.

Nhờ cơ chế LIFO, việc cấp phát và giải phóng bộ nhớ trên Stack có thể được thực hiện rất hiệu quả.

Activation Record Mỗi lời gọi hàm trên Stack được biểu diễn bởi một cấu trúc dữ liệu gọi là *bản ghi kích hoạt (Activation Record)* hay còn gọi là *Stack Frame*.

Activation Record thường chứa các thành phần sau:

- biến cục bộ (*Local Variables*),
- kết quả trung gian (*Intermediate Results*),
- các con trỏ liên kết với các frame khác trên Stack (*Dynamic Chain Pointer* hoặc *Static Chain Pointer*).

đối với bản ghi của Hàm / Thủ tục sẽ có thêm:

- địa chỉ trả về sau khi hàm kết thúc (*Return Address*),
- địa chỉ của kết quả trả về (*Address for Result*),
- tham số của hàm (*Parameters*),

Activation Record (Block)

Dynamic Chain Pointer
Local Variables
Intermediate Results

Activation Record (Procedure)

Dynamic Chain Pointer
Static Chain Pointer
Return Address
Address for Result
Parameters
Local Variables
Intermediate Results

Hình 2: So sánh cấu trúc Activation Record cho Block và Procedure

Dynamic Chain Trong Stack, các Activation Record được liên kết với nhau thông qua một chuỗi gọi là *chuỗi động (Dynamic Chain)*.

Dynamic Chain được tạo ra dựa trên thứ tự lời gọi hàm trong thời gian chạy. Mỗi Activation Record sẽ lưu một con trỏ đến Activation Record của hàm đã gọi nó.

Chuỗi liên kết này giúp hệ thống runtime:

- khôi phục trạng thái thực thi khi hàm kết thúc,
- quay trở lại đúng vị trí trong chương trình sau khi hoàn thành lời gọi hàm.

Static Chain Bên cạnh Dynamic Chain, nhiều hệ thống runtime còn sử dụng một cấu trúc gọi là *chuỗi tĩnh (Static Chain)*.

Static Chain phản ánh cấu trúc lồng nhau của các khối chương trình trong mã nguồn. Thông qua chuỗi này, hệ thống runtime có thể truy cập các biến nằm trong các phạm vi bao ngoài.

Cơ chế này liên quan trực tiếp đến cách một ngôn ngữ lập trình hiện thực hóa *Phạm vi tĩnh (Static scope)*.

Nói cách khác:

- Dynamic Chain phản ánh **chuỗi lời gọi hàm trong runtime**,
- Static Chain phản ánh **cấu trúc tĩnh của chương trình**.

Sự phân biệt này đóng vai trò quan trọng trong việc hiểu cách hệ thống runtime xác định biến nào sẽ được truy cập tại một vị trí cụ thể trong chương trình.

Cơ chế quản lý bộ nhớ trên Stack Khi các biến cục bộ và cấu trúc dữ liệu trong một chương trình có thể được xác định kích thước tại thời điểm biên dịch, trình biên dịch có thể tính toán trước kích thước của mỗi *Activation Record*. Điều này cho phép hệ thống runtime tổ chức bộ nhớ theo một cách rất hiệu quả.

Về bản chất, mỗi khối lệnh hoặc hàm/thủ tục trong chương trình sẽ tương ứng với một *Activation Record*. Khi chương trình được thực thi, các bản ghi này không được tạo ra một cách ngẫu nhiên mà tuân theo cấu trúc lồng nhau của chương trình.

Một đặc điểm quan trọng là: nếu một khối con nằm trong một khối cha, thì khối con luôn được khởi tạo sau nhưng sẽ kết thúc trước. Quy tắc này hoàn toàn phù hợp với nguyên lý *Last-In First-Out (LIFO)* của Stack.

Do đó, Stack không chỉ đơn thuần là nơi lưu trữ dữ liệu, mà còn đóng vai trò như một cơ chế biểu diễn trạng thái thực thi của chương trình:

- Khi một khối hoặc hàm được bắt đầu, một *Activation Record* mới sẽ được đẩy vào Stack.
- *Activation Record* này chỉ có thể tồn tại nếu *Activation Record* của khối bao ngoài vẫn còn tồn tại.
- Khi khối kết thúc hoặc hàm trả về, *Activation Record* tương ứng sẽ bị loại bỏ khỏi Stack.

Nói cách khác, sự tồn tại của một *Activation Record* trong Stack chính là biểu diễn trực tiếp cho việc khối hoặc hàm đó đang hoạt động trong thời gian chạy.

Điều này cũng giải thích vì sao:

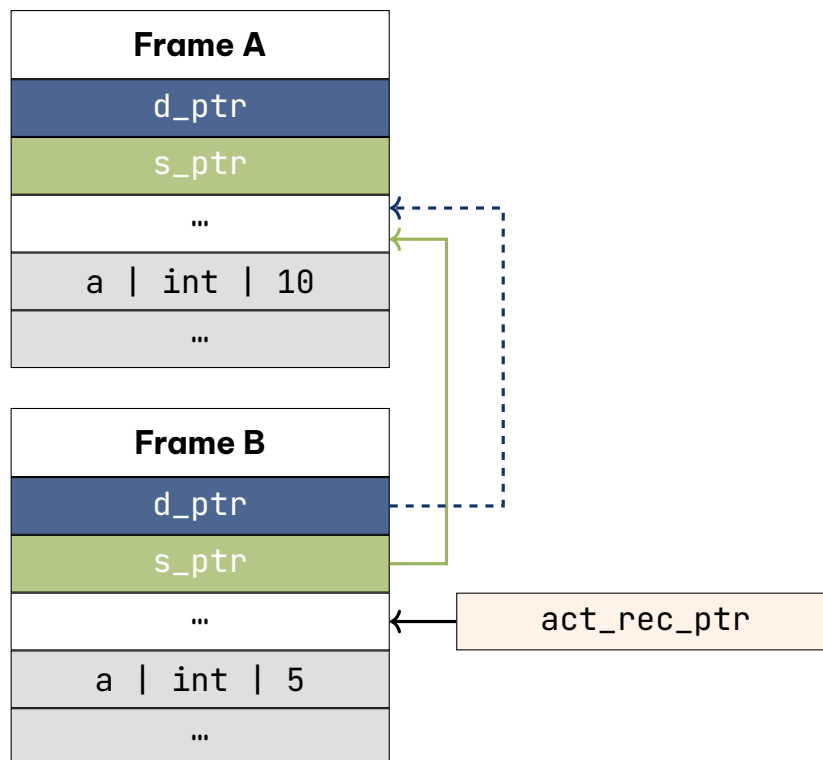
- một hàm chỉ tồn tại trong Stack khi có lời gọi,
- và sẽ bị loại bỏ ngay sau khi hoàn thành thực thi,
- các biến cục bộ chỉ tồn tại trong phạm vi của *Activation Record* tương ứng.

Nhờ cơ chế này, Stack vừa đảm bảo việc cấp phát bộ nhớ hiệu quả, vừa phản ánh chính xác cấu trúc thực thi của chương trình.

Cấu trúc tổ chức các Activation Record trên Stack Ví dụ xét mã nguồn sau:

```
1 // frame A
2 {
3     int a = 10;
4     // frame B
5     {
6         int a = 5;
7     }
8 }
```

Cách các *Activation Record* được tổ chức và liên kết với nhau trong Stack như sau:



Hình 3: Minh họa cấu trúc tổ chức các Activation Record trên Stack

Trước hết, các bản ghi không tồn tại độc lập mà được đưa vào Stack theo đúng thứ tự xuất hiện của các khối lệnh hoặc lời gọi hàm/thủ tục trong quá trình thực thi. Khi một khối mới được bắt đầu, một Activation Record tương ứng sẽ được đẩy lên đầu Stack; khi khối đó kết thúc, bản ghi sẽ bị loại bỏ khỏi Stack.

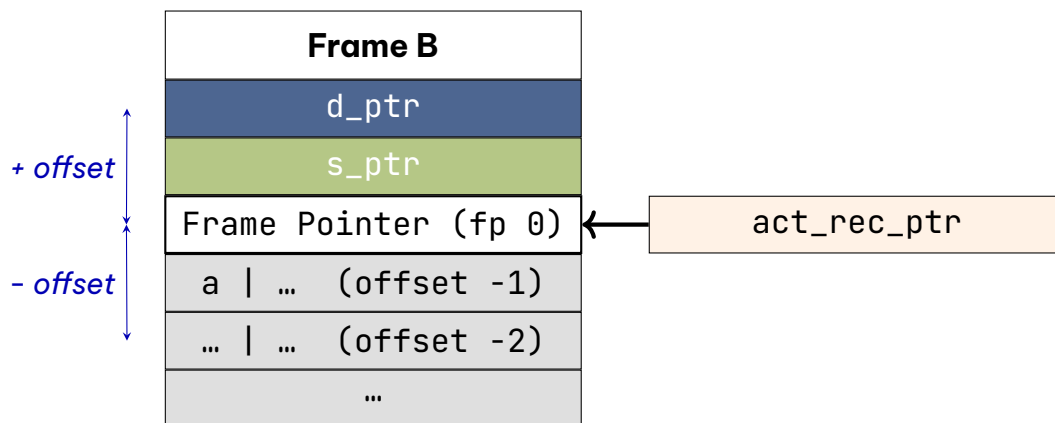
Để quản lý cấu trúc này, hệ thống runtime duy trì một con trỏ gọi là *Activation Record Pointer* (act_rec_ptr). Con trỏ này luôn trỏ tới bản ghi ở đầu Stack, tức là bản ghi của khối hoặc hàm đang được thực thi tại thời điểm hiện tại.

Mỗi Activation Record ngoài việc lưu trữ dữ liệu còn chứa các con trỏ liên kết:

- *Dynamic Chain Pointer* (*d_ptr*): trỏ tới Activation Record của khối/hàm đã kích hoạt khối hiện tại (theo thứ tự runtime),
- *Static Chain Pointer* (*s_ptr*): trỏ tới Activation Record của khối bao ngoài gần nhất (theo cấu trúc chương trình).

Trong trường hợp đơn giản như hình minh họa (block lồng trực tiếp), cả hai con trỏ này đều trỏ tới cùng một Activation Record của khối cha. Tuy nhiên, trong các trường hợp phức tạp hơn (ví dụ như lời gọi hàm không cùng mức lồng), hai con trỏ này có thể trỏ tới các bản ghi khác nhau.

Một điểm quan trọng trong thiết kế là: các con trỏ này thường không trỏ tới đầu hoặc cuối bản ghi, mà trỏ tới một vị trí mốc cố định bên trong Activation Record (thường gọi là *frame pointer*). Từ vị trí này, hệ thống runtime có thể truy cập các thành phần khác của bản ghi thông qua cơ chế *offset* (dịch chuyển tương đối).



Địa chỉ thực tế = fp + offset
 (Truy cập trực tiếp, không cần tìm kiếm tên biến)

Hình 4: Cơ chế truy cập dữ liệu qua Frame Pointer và Offset trong Frame B

Nhờ đó, việc truy cập dữ liệu trong Stack trở nên rất hiệu quả:

- không cần tìm kiếm theo tên biến tại runtime,
- chỉ cần tính toán địa chỉ dựa trên offset cố định,
- các thao tác cấp phát và giải phóng đều diễn ra theo cơ chế LIFO đơn giản.

Chính vì vậy, so với Heap, việc quản lý bộ nhớ trên Stack đơn giản hơn đáng kể và gần như không cần các chiến lược tối ưu phức tạp.

1.1.3 Bộ nhớ động và Heap

Bộ nhớ động Không phải mọi đối tượng dữ liệu trong chương trình đều có thể xác định kích thước và vòng đời tại thời điểm biên dịch. Trong nhiều trường hợp, chương

trình cần tạo ra các đối tượng mới trong quá trình chạy, với kích thước hoặc thời gian tồn tại phụ thuộc vào dữ liệu đầu vào hoặc luồng thực thi.

Những đối tượng như vậy được quản lý thông qua cơ chế *cấp phát động (dynamic allocation)*.

Trong ngôn ngữ C++, việc cấp phát bộ nhớ động thường được thực hiện thông qua toán tử `new`, và giải phóng thông qua `delete`.

```
1 int* p = new int;    // allocate in Heap
2 *p = 10;
3
4 delete p;           // deallocate
```

Các đối tượng được tạo ra theo cách này không nằm trên Stack mà được lưu trữ trong vùng bộ nhớ gọi là **Heap**. Vòng đời của chúng không phụ thuộc trực tiếp vào lời gọi hàm mà phụ thuộc vào thời điểm chương trình chủ động giải phóng bộ nhớ.

Vì sao Stack không thể quản lý bộ nhớ động Stack hoạt động theo nguyên lý LIFO, nghĩa là vùng nhớ được cấp phát sau cùng sẽ được giải phóng trước. Cơ chế này rất hiệu quả đối với các biến cục bộ trong lời gọi hàm, khi thứ tự vào/ra của các khối tuân thủ chặt chẽ cấu trúc chương trình.

Tuy nhiên, có hai hạn chế quan trọng khiến Stack không thể quản lý bộ nhớ động.

Thứ nhất, trong trường hợp có đệ quy, số lần lời gọi hàm không thể xác định tại thời điểm biên dịch. Mỗi lần gọi hàm sẽ tạo ra một *Activation Record* mới trên Stack, nhưng số lượng các bản ghi này phụ thuộc vào dữ liệu đầu vào và chỉ được xác định trong quá trình thực thi.

```
1 // recursive function
2 int factorial(int n) {
3     if (n <= 1) return 1;
4     return n * factorial(n - 1);
5 }
```

Trong ví dụ trên, số lượng Activation Record được tạo ra phụ thuộc vào giá trị của `n`. Do đó, không thể cấp phát trước toàn bộ bộ nhớ cần thiết theo cách tĩnh trong quá trình biên dịch.

Thứ hai, đối với nhiều cấu trúc dữ liệu, vòng đời của các đối tượng không tuân theo thứ tự LIFO. Ví dụ, trong các cấu trúc như danh sách liên kết hoặc cây, các phần tử có thể được tạo ra và giải phóng theo nhiều thứ tự khác nhau.

```
1 // struct type node
2 struct Node {
3     int value;
4     Node* next;
5 };
6
```

```

7 Node* head = new Node{1, nullptr};
8 head->next = new Node{2, nullptr};
9
10 // it is possible to deallocate second pointer
11 delete head->next;
12
13 // then, deallocate first pointer
14 delete head;

```

Trong trường hợp này, các đối tượng được cấp phát và giải phóng theo thứ tự không dự đoán trước và không tuân theo LIFO.

Vì vậy, Stack không thể quản lý hiệu quả các đối tượng như vậy. Đây chính là lý do tồn tại của Heap – một cơ chế cho phép cấp phát và giải phóng bộ nhớ một cách linh hoạt trong thời gian chạy.

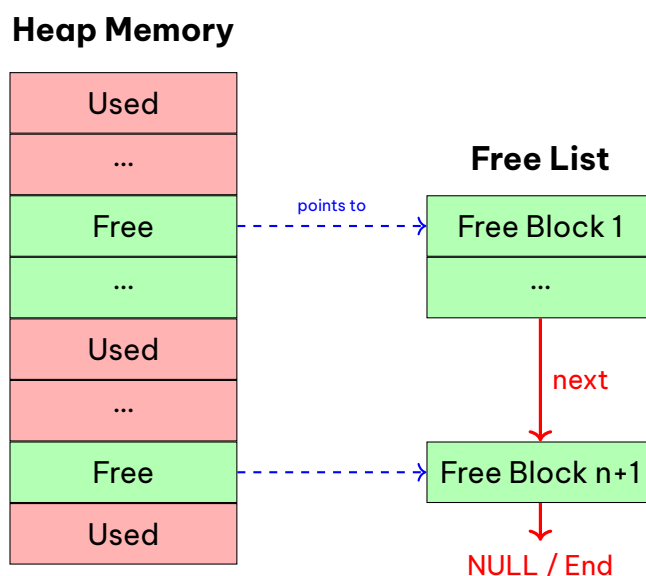
Quản lý bộ nhớ trong Heap Khác với Stack, Heap là một vùng bộ nhớ cho phép cấp phát và giải phóng các khối dữ liệu với kích thước và thời điểm linh hoạt.

Một cách đơn giản để quản lý Heap là duy trì một danh sách các khối bộ nhớ chưa được sử dụng, thường được gọi là *danh sách tự do (free list)*.

Mỗi khi chương trình yêu cầu cấp phát bộ nhớ, hệ thống runtime sẽ:

- tìm một khối bộ nhớ đủ lớn trong free list,
- cấp phát khối đó cho chương trình,
- và cập nhật lại danh sách các khối còn trống.

Khi bộ nhớ được giải phóng, khối bộ nhớ tương ứng sẽ được đưa trở lại free list để có thể tái sử dụng.

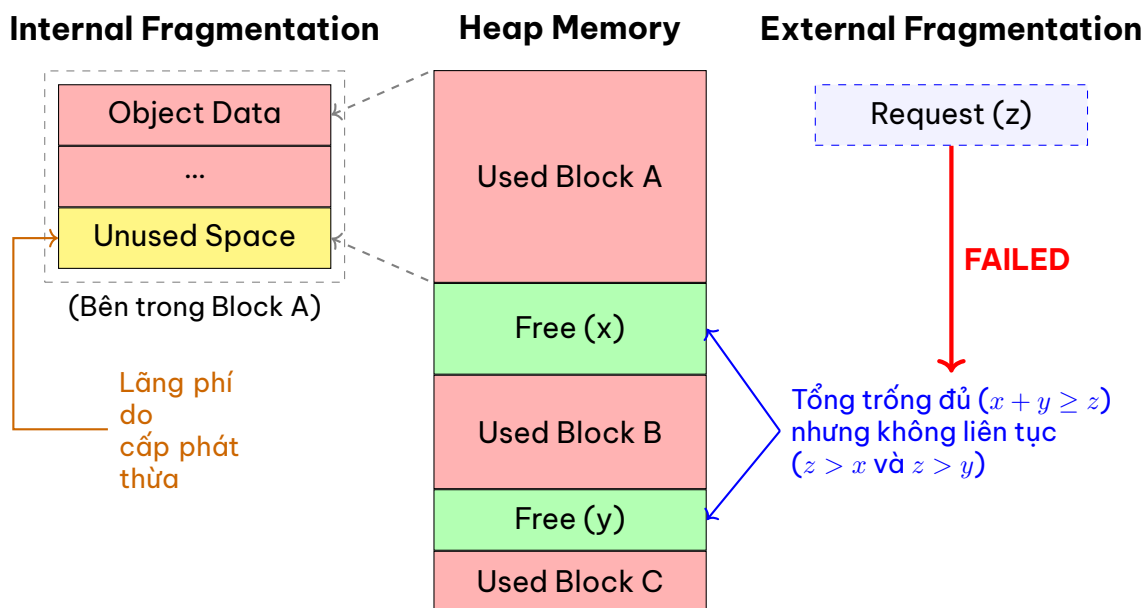


Hình 5: Minh họa các khối bộ nhớ trong Heap và danh sách free list

Fragmentation Mặc dù Heap cho phép cấp phát bộ nhớ linh hoạt hơn so với Stack, nhưng cơ chế này cũng dẫn đến một vấn đề quan trọng gọi là *phân mảnh bộ nhớ (fragmentation)*.

Có hai dạng phân mảnh phổ biến:

- **Internal fragmentation** - khối bộ nhớ được cấp phát lớn hơn nhu cầu thực tế của đối tượng.
- **External fragmentation** - bộ nhớ trống bị chia thành nhiều khối nhỏ rời rạc, khiến việc cấp phát một khối lớn trở nên khó khăn.



Hình 6: Minh họa Phân mảnh nội vi (Internal) và Phân mảnh ngoại vi (External)

Phân mảnh là một trong những thách thức quan trọng trong thiết kế các hệ thống quản lý bộ nhớ động.

Liên hệ với chương trình C++ Trong C++, các đối tượng được cấp phát trên Heap thường được truy cập thông qua con trỏ. Điều này dẫn đến một sự phân tách quan trọng trong mô hình bộ nhớ của chương trình:

- con trỏ thường nằm trên Stack,
- trong khi dữ liệu mà con trỏ tham chiếu tới nằm trên Heap.

Sự kết hợp giữa Stack và Heap cho phép chương trình vừa tận dụng được hiệu quả của Stack trong quản lý lời gọi hàm, vừa có thể tạo ra các cấu trúc dữ liệu linh hoạt trong runtime.

1.1.4 Xử lý Phạm vi

Phạm vi của biến Trong một chương trình, mỗi biến không chỉ có một vị trí trong bộ nhớ mà còn có một phạm vi truy cập xác định. Như đã giới thiệu trong phần lý thuyết, hai mô hình phổ biến để xác định phạm vi của biến là *Phạm vi tĩnh* (Static scope) và *Phạm vi động* (Dynamic scope).

Sự khác biệt giữa hai mô hình này nằm ở cách hệ thống runtime xác định đối tượng nào sẽ được liên kết với một tên biến tại thời điểm chương trình thực thi.

- Trong **Phạm vi tĩnh**, biến được xác định dựa trên cấu trúc tĩnh của chương trình.
- Trong **Phạm vi động**, biến được xác định dựa trên chuỗi lời gọi hàm trong thời gian chạy.

Các cơ chế quản lý bộ nhớ đã được giới thiệu trước đó – đặc biệt là cấu trúc Stack và Activation Record – đóng vai trò quan trọng trong việc hiện thực hóa hai mô hình phạm vi này.

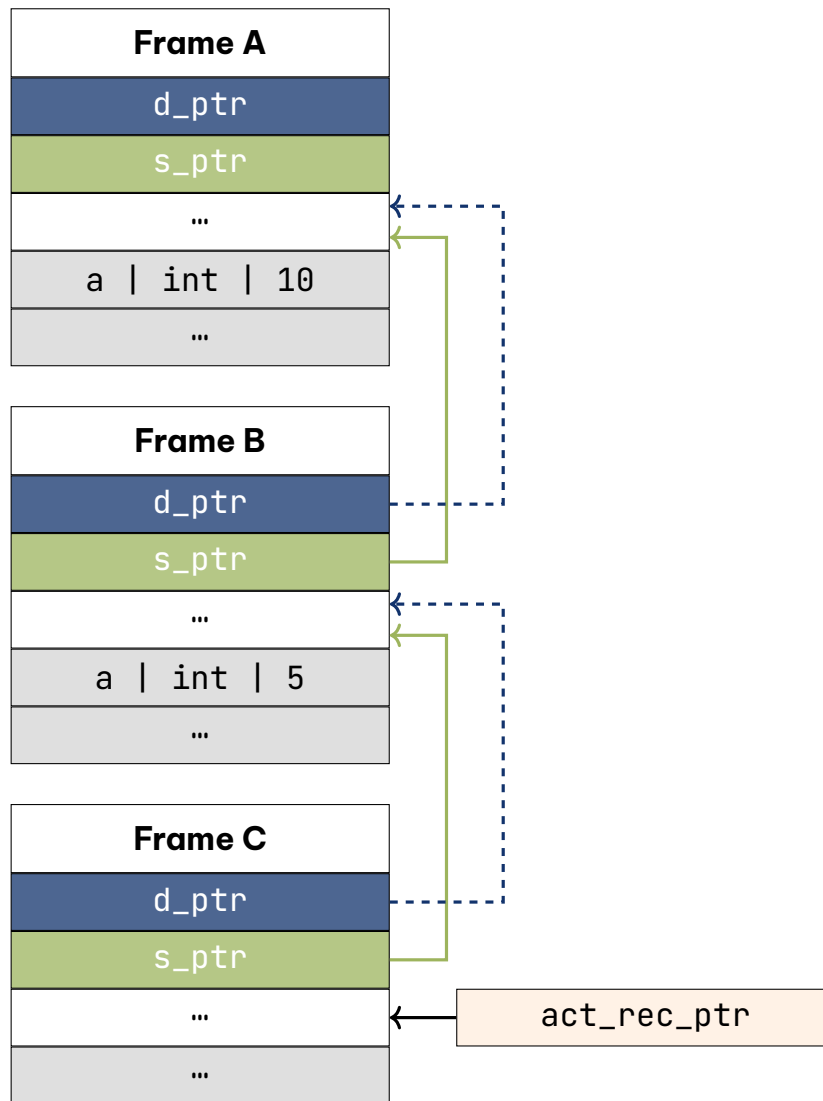
Cài đặt Phạm vi tĩnh Trong các ngôn ngữ lập trình hiện đại như C++, phạm vi của biến được xác định theo mô hình phạm vi tĩnh. Điều này có nghĩa là trình biên dịch có thể xác định mối quan hệ giữa các khối chương trình ngay từ khi phân tích mã nguồn.

Để hiện thực hóa cơ chế này trong runtime, hệ thống thường sử dụng *chuỗi tĩnh* (Static Chain) đã được lưu trong các Activation Record.

Mỗi Activation Record sẽ chứa một con trỏ tới Activation Record của khối bao ngoài gần nhất. Khi chương trình cần truy cập một biến nằm ngoài phạm vi cục bộ, hệ thống runtime có thể lần theo Static Chain để tìm Activation Record tương ứng. Ví dụ xét mã nguồn sau:

```
1 // frame A
2 {
3     int a = 10;
4     // frame B
5     {
6         int a = 5;
7         // frame C
8         {
9             a = a + 1;
10        }
11    }
12 }
```

Chuỗi tĩnh được xử lý trong Stack có thể được biểu diễn như sau:



Hình 7: Minh họa Static Chain (đường nét liền) và Dynamic Chain (đường nét đứt) giữa các Activation Record (Phạm vi Tĩnh)

Một điểm quan trọng là: các biến cục bộ như `a` trong Frame B được lưu trực tiếp trong Activation Record tương ứng.

Nhờ cơ chế *offset*, trình biên dịch đã xác định trước vị trí của biến so với một mốc cố định (thường là *frame pointer*). Khi thực thi, hệ thống chỉ cần lấy địa chỉ bản ghi hiện tại và cộng/trừ *offset* để truy cập trực tiếp, không cần tìm kiếm theo tên.

Do đó, việc truy cập biến cục bộ có chi phí hằng số và rất hiệu quả.

Trong trường hợp cần truy cập biến không-cục-bộ, ví dụ `a` từ Frame C, hệ thống runtime sẽ lần theo *Static Chain Pointer* (`s_ptr`) để tới Frame B, sau đó tiếp tục dùng *offset* để truy cập giá trị.

Tuy nhiên, cách làm này có chi phí phụ thuộc vào độ sâu mức lồng:

- phải lần theo nhiều con trỏ,
- không đảm bảo thời gian truy cập hằng số.

Vì vậy, trong các chương trình lồng sâu, việc truy cập qua Static Chain có thể kém hiệu quả. Một phương pháp tối ưu hơn là sử dụng *vector hiển thị (Display)*, cho phép truy cập trực tiếp tới Activation Record theo từng mức lồng.

Cài đặt Phạm vi động Trong mô hình **Phạm vi động**, việc xác định biến phụ thuộc vào chuỗi lời gọi hàm tại thời điểm chương trình chạy. Điều này có nghĩa là hệ thống runtime cần tìm kiếm biến dựa trên *chuỗi động (Dynamic Chain)* của các Activation Record.

Một cách đơn giản là lần ngược theo Dynamic Chain trên Stack để tìm biến phù hợp. Tuy nhiên, việc tìm kiếm tuần tự như vậy có thể làm giảm hiệu năng của chương trình. Ví dụ với mã nguồn sau:

```
1 // frame A
2 {
3     int a = 10;
4     // frame B
5     {
6         // frame foo
7         void foo() {
8             int b = 1;
9             a = a + b;
10        }
11
12        int a = 5;
13        // frame C
14        {
15            foo();
16        }
17    }
18 }
```

Để thấy rõ cơ chế này, xét lời gọi hàm foo trong ví dụ trên. Bên trong foo, biến a không được khai báo cục bộ, nên được xem là một biến không-cục-bộ (*non-local*).

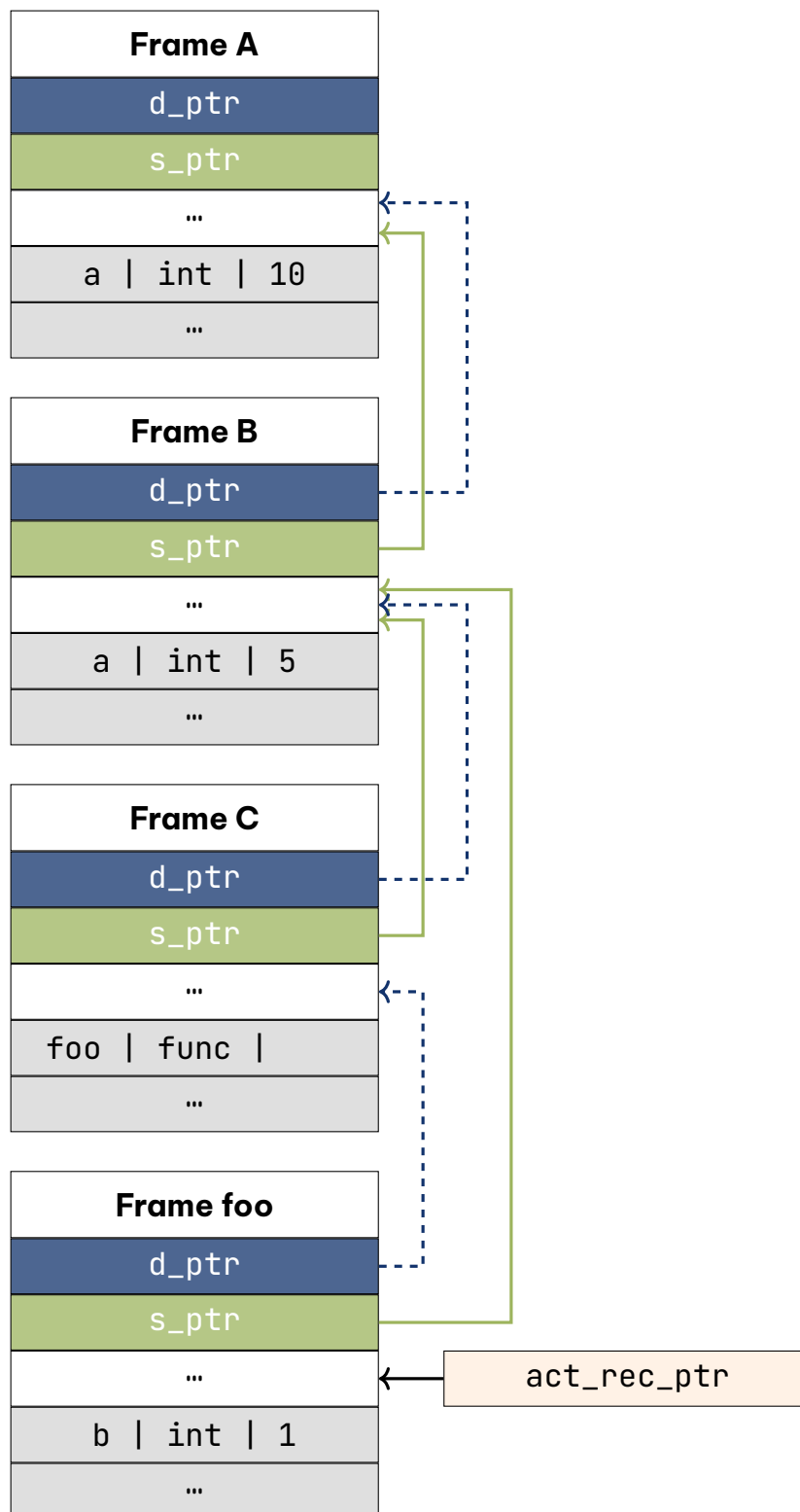
Để xác định a thuộc về Activation Record nào, hệ thống runtime cần dựa vào các con trỏ liên kết giữa các bản ghi:

- Nếu đi theo *Dynamic Chain Pointer (d_ptr)*, hệ thống sẽ lần theo chuỗi lời gọi hàm thực tế trong runtime.
- Nếu đi theo *Static Chain Pointer (s_ptr)*, hệ thống sẽ lần theo cấu trúc lồng nhau của chương trình trong mã nguồn.

Mỗi bước di chuyển theo các con trỏ này tương ứng với việc mở rộng *môi trường (environment)* của bản ghi hiện tại, cho phép truy cập đến các biến không-cục-bộ ở các phạm vi trước đó.

Do đó, việc lựa chọn đi theo chuỗi nào sẽ quyết định biến a được ánh xạ tới bản

ghi nào trong Stack. Để trực quan hơn, ở ví dụ trên, bên trong Stack có thể biểu diễn chuỗi tĩnh và chuỗi động như sau:



Hình 8: Minh họa Static Chain (đường nét liền) và Dynamic Chain (đường nét đứt) giữa các Activation Record (Phạm vi Động)

Từ hình minh họa, có thể thấy rằng để xác định một biến không-cục-bộ, hệ thống runtime cần thực hiện việc tìm kiếm ngược qua các Activation Record trước đó thông

qua các chuỗi liên kết.

Trong trường hợp sử dụng *Dynamic Chain*, việc tìm kiếm sẽ diễn ra theo thứ tự lời gọi hàm, tức là phải lần lượt duyệt qua từng bản ghi trên Stack cho đến khi tìm thấy tên phù hợp.

Tuy nhiên, cách tiếp cận này tồn tại nhiều hạn chế về hiệu năng:

- số bước tìm kiếm phụ thuộc vào độ sâu của Stack tại thời điểm runtime,
- có thể phải duyệt qua nhiều bản ghi không liên quan,
- không thể xác định trước chi phí truy cập tại thời điểm biên dịch.

Chính vì vậy, trong thực tế, việc chỉ dựa vào việc duyệt ngược Stack thường không hiệu quả. Vì lý do đó, một số hệ thống runtime sử dụng các cấu trúc dữ liệu hỗ trợ như:

- *Association List (A-list)* – danh sách các cặp tên–giá trị đang hoạt động trong runtime.
- *Central Reference Table (CRT)* – bảng tra cứu trung tâm để xác định giá trị hiện tại của các biến.

Các cấu trúc này giúp hệ thống runtime tìm kiếm biến nhanh hơn so với việc duyệt toàn bộ Stack.

Liên hệ với mô hình bộ nhớ trong C++ Trong C++, phạm vi của biến được xác định theo mô hình phạm vi tĩnh. Điều này có nghĩa là trình biên dịch có thể xác định trước mối quan hệ giữa các khối chương trình.

Trong quá trình thực thi:

- các biến cục bộ được lưu trữ trong Activation Record trên Stack,
- các biến toàn cục hoặc biến tĩnh được lưu trong Static Storage,
- các đối tượng được cấp phát động nằm trong Heap.

Đặc biệt, khi chương trình sử dụng bộ nhớ động, các đối tượng trên Heap thường được truy cập thông qua con trỏ nằm trên Stack. Vì vậy, mặc dù dữ liệu thực tế nằm trong Heap, việc quản lý luồng thực thi và phạm vi của các biến vẫn chủ yếu diễn ra trên Stack.

Sự kết hợp giữa Stack, Heap và cơ chế phạm vi tĩnh giúp các ngôn ngữ như C++ có thể quản lý dữ liệu hiệu quả trong cả thời điểm biên dịch và thời gian chạy.

1.2 Stack trong C++

Trong C++, phần lớn các biến cục bộ được lưu trữ trên Stack. Các biến này được tạo ra khi chương trình đi vào một khối lệnh hoặc hàm và bị hủy khi khối đó kết thúc. Cơ chế quản lý tài nguyên theo vòng đời của đối tượng này được gọi là *Khởi tạo gắn với thu hồi tài nguyên (Resource Acquisition Is Initialization – RAII)*.

Ý tưởng chính của RAII là gắn việc cấp phát và giải phóng tài nguyên vào *vòng đời (Lifetime)* của một đối tượng. Khi đối tượng được tạo ra, tài nguyên cần thiết sẽ được cấp phát trong hàm khởi tạo. Khi đối tượng rời khỏi phạm vi, hàm hủy sẽ tự động được gọi để giải phóng tài nguyên.

Nhờ cơ chế này, C++ có thể quản lý tài nguyên một cách an toàn mà không cần lập trình viên phải nhớ giải phóng tài nguyên một cách thủ công.

```
1 #include <iostream>
2
3 class FileHandler {
4 public:
5     FileHandler() {
6         std::cout << "Open resource\n";
7     }
8
9     ~FileHandler() {
10        std::cout << "Release resource\n";
11    }
12 };
13
14 void example() {
15     FileHandler file; // resource acquired
16 } // destructor called automatically here
```

Trong ví dụ trên, đối tượng file được tạo trên Stack. Khi hàm example() kết thúc, đối tượng rời khỏi phạm vi và hàm hủy được gọi tự động để giải phóng tài nguyên.

Cơ chế RAII là nền tảng cho nhiều thư viện trong C++, bao gồm quản lý file, mutex trong lập trình song song, và các cấu trúc quản lý bộ nhớ an toàn.

1.3 Heap trong C++

Bên cạnh Stack, C++ còn cho phép cấp phát bộ nhớ động trên Heap. Các đối tượng trên Heap có *vòng đời (Lifetime)* độc lập với phạm vi của khối chương trình, và do đó cần được quản lý thông qua *con trỏ (pointer)*.

Con trỏ là một biến lưu trữ địa chỉ của một đối tượng trong bộ nhớ. Khi sử dụng cấp phát động, lập trình viên cần sử dụng toán tử new để tạo đối tượng trên Heap và delete để giải phóng bộ nhớ.

```
1 int* p = new int(10); // allocate memory on Heap
2 std::cout << *p << std::endl;
3
4 delete p; // free memory
```

Trong ví dụ trên:

- new int(10) tạo một đối tượng trên Heap

- biến p (trên Stack) lưu địa chỉ của đối tượng đó
- toán tử $*p$ được sử dụng để truy cập giá trị

Nếu bộ nhớ được cấp phát bằng `new` nhưng không được giải phóng bằng `delete`, chương trình sẽ gặp hiện tượng rò rỉ bộ nhớ (*Memory Leak*). Đây là một trong những lỗi phổ biến trong các chương trình sử dụng cấp phát động.

Vì lý do này, các phiên bản C++ hiện đại khuyến khích sử dụng các cơ chế quản lý bộ nhớ an toàn hơn thay vì thao tác trực tiếp với `new` và `delete`.

1.4 Mảng trong C++

Trong C++, một *mảng* (*Array*) là một tập hợp các phần tử cùng kiểu dữ liệu được lưu trữ liên tiếp trong bộ nhớ. Nhờ đặc điểm này, chương trình có thể truy cập trực tiếp tới bất kỳ phần tử nào thông qua việc tính toán vị trí của nó trong bộ nhớ.

Ví dụ:

```
1 int a[5] = {10, 20, 30, 40, 50};
```

Trong bộ nhớ, các phần tử của mảng được lưu liên tiếp:

a[0]	a[1]	a[2]	a[3]	a[4]
------	------	------	------	------

Giả sử địa chỉ của `a[0]` là `base`. Khi đó, địa chỉ của phần tử thứ i có thể được tính bằng công thức:

$$\text{address}(a[i]) = \text{base} + i \times \text{sizeof}(\text{int})$$

Trong C++, cơ chế này được ẩn phía sau cú pháp truy cập mảng. Biểu thức

```
1 a[i]
```

thực chất tương đương với

```
1 *(a + i)
```

Trong đó:

- `a` là con trỏ tới phần tử đầu tiên của mảng
- phép toán cộng con trỏ sẽ tự động dịch chuyển theo kích thước của kiểu dữ liệu

Nhờ cơ chế này, việc truy cập phần tử của mảng có thể được thực hiện với chi phí thời gian hằng số (*constant time*).

I.5 Ownership trong C++

Một khái niệm quan trọng trong C++ hiện đại là *quyền sở hữu tài nguyên (Ownership)*. Ý tưởng của Ownership là mỗi tài nguyên trong chương trình nên có một đối tượng chịu trách nhiệm quản lý vòng đời của nó.

Để hỗ trợ mô hình này, thư viện chuẩn của C++ cung cấp các *con trỏ thông minh (Smart Pointer)*. Một trong những loại phổ biến nhất là `unique_ptr`, đại diện cho *quyền sở hữu duy nhất (Unique Ownership)* đối với một đối tượng trên Heap.

```
1 #include <memory>
2
3 std::unique_ptr<int> p = std::make_unique<int>(10);
4
5 std::cout << *p << std::endl;
```

Trong ví dụ này:

- `std::unique_ptr` quản lý một đối tượng trên Heap
- đối tượng sẽ được giải phóng tự động khi `unique_ptr` rời khỏi phạm vi
- không cần gọi `delete`

Một đặc điểm quan trọng của `unique_ptr` là nó không thể được sao chép. Quyền sở hữu chỉ có thể được *chuyển giao (move)* sang một đối tượng khác.

```
1 auto p1 = std::make_unique<int>(5);
2 auto p2 = std::move(p1);
```

Sau khi thực hiện `std::move`, quyền sở hữu đối tượng được chuyển từ `p1` sang `p2`.

Mô hình Ownership giúp giảm thiểu nhiều lỗi phổ biến trong quản lý bộ nhớ như rò rỉ bộ nhớ, giải phóng bộ nhớ nhiều lần, hoặc truy cập vào vùng nhớ đã bị hủy. Vì vậy, việc sử dụng các Smart Pointer như `unique_ptr` được xem là một thực hành tốt trong lập trình C++ hiện đại.

I.6 Mục tiêu

Trong bài thực hành tuần này, các bạn sẽ tìm hiểu cách *Quản lý bộ nhớ (Memory Management)* được hiện thực trong chương trình C++ thông qua các vùng nhớ như Stack, Heap và Static Storage. Từ đó, các bạn sẽ quan sát và thực hành cách dữ liệu được cấp phát, truy cập và giải phóng trong suốt *Vòng đời (Lifetime)* của chương trình.

Thông qua việc sử dụng con trỏ và các cơ chế cấp phát bộ nhớ, bài học giúp các bạn kết nối giữa mô hình lý thuyết của *Máy trừu tượng (Abstract Machine)* và cách chương trình thực thi trong thực tế. Cụ thể, các bạn sẽ cần đạt được các mục tiêu sau:

- Hiểu được **vai trò của Quản lý bộ nhớ** (*Memory Management*) trong việc kiểm soát *Vòng đời* (*Lifetime*) và *Khả năng truy cập* (*Accessibility*) của dữ liệu.
- Phân biệt và nhận diện được **ba vùng bộ nhớ chính** trong chương trình:
 - Static Storage (data segment),
 - Stack,
 - Heap,
 thông qua việc quan sát địa chỉ bộ nhớ trong runtime.
- Hiểu **cơ chế hoạt động của Stack** thông qua *Activation Record*, quy tắc LIFO và mối liên hệ với *Phạm vi tĩnh* (*Static scope*) và *Phạm vi động* (*Dynamic scope*).
- Hiểu được **bản chất của Heap** và lý do tồn tại của bộ nhớ động, bao gồm các vấn đề như cấp phát linh hoạt và phân mảnh bộ nhớ.
- Sử dụng **con trỏ** để:
 - lưu trữ và thao tác với địa chỉ bộ nhớ,
 - truy cập dữ liệu trên Heap,
 - làm việc với mảng thông qua pointer arithmetic.
- Hiểu **mối quan hệ giữa kiểu dữ liệu và con trỏ**, đặc biệt là vai trò của `sizeof` trong việc xác định cách truy cập bộ nhớ.
- Thực hành cấp phát và giải phóng bộ nhớ bằng:
 - `malloc/free` (kiểu C),
 - `new/delete` (kiểu C++),
 và nhận biết các lỗi phổ biến như memory leak và dangling pointer.
- Hiểu và áp dụng đúng thứ tự giải phóng bộ nhớ trong các **cấu trúc con trỏ lồng nhau**.
- Quan sát và phân tích cách mảng được lưu trữ liên tiếp trong bộ nhớ, và cách con trỏ đóng vai trò như một “cửa sổ” để truy cập vùng nhớ này.
- Nhận thức được hạn chế của việc quản lý bộ nhớ thủ công, từ đó chuẩn bị cho các cơ chế an toàn hơn trong C++ hiện đại như **RAII** và **smart pointer**.

II. Bài tập thực hành

Bài tập 1: Con trỏ và bộ nhớ trên Stack

Mô tả:

Xét chương trình sau:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int *p;
7
8     cout << "p (uninitialized): " << p << endl;
9
10    {
11        int x = 5;
12        p = &x;
13        cout << "&x: " << &x << ", *p: " << *p << endl;
14    }
15
16    {
17        int y = 100;
18        cout << "address y: " << &y << endl;
19    }
20
21    cout << "p: " << p << endl;
22    cout << "*p: " << *p << endl;
23 }
```

Yêu cầu:

1. Giải thích giá trị của p và *p tại từng thời điểm in ra.
2. Sau khi block chứa biến x kết thúc, con trỏ p có còn hợp lệ không? Vì sao?
3. Dự đoán mối quan hệ giữa địa chỉ của x và y. Trong nhiều hệ thống (ví dụ Linux), hai địa chỉ này có thể như thế nào?
4. Dự đoán giá trị của *p ở cuối chương trình. Liệu chương trình có luôn in ra cùng một kết quả không?

Câu hỏi thảo luận:

1. Biến *x* có *vòng đời (Lifetime)* như thế nào? Khi nào nó bị giải phóng khỏi Stack?
2. Sau khi *x* bị hủy, con trỏ *p* trở thành loại con trỏ gì?
3. Vì sao trong block thứ hai, biến *y* có thể được cấp phát tại cùng vị trí bộ nhớ với *x* trước đó?
4. Điều gì sẽ xảy ra nếu ta tiếp tục sử dụng **p*? Hành vi này có được đảm bảo bởi chuẩn C++ hay không?

Bài tập 2: Con trỏ và bộ nhớ Heap

Mô tả:

Xét chương trình sau:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int *p;
7     {
8         int *x = new int(5);
9         p = x;
10        cout << "x address: " << x << " value: " << *x << endl;
11        delete x;
12    }
13
14    {
15        int *y = new int(100);
16        cout << "y address: " << y << " value: " << *y << endl;
17    }
18
19    cout << "p: " << p << endl;
20    cout << "*p: " << *p << endl;
21 }
```

Yêu cầu:

1. Chương trình trên có hợp lệ về mặt cú pháp không? Có tồn tại lỗi logic nào không?
2. Dự đoán kết quả in ra của chương trình (giải thích rõ từng bước).

3. Giá trị của *p ở cuối chương trình có xác định hay không? Vì sao?

Câu hỏi thảo luận:

1. Sau khi delete x, con trỏ p đang trỏ tới đâu? Trạng thái của vùng nhớ đó là gì?
2. Nếu vùng nhớ vừa bị giải phóng được cấp phát lại cho y, điều gì có thể xảy ra với *p?
3. So sánh hiện tượng này với trường hợp biến trên Stack (đã học ở bài trước).
4. Đây là ví dụ của loại lỗi gì trong C++? (gợi ý: liên quan tới con trỏ treo).

Bài tập 3: Kiểu dữ liệu của con trỏ và phép toán địa chỉ

Mô tả:

Xét chương trình sau:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("sizeof(char*)   = %zu bytes\n", sizeof(char *));
6     printf("sizeof(int*)    = %zu bytes\n", sizeof(int *));
7     printf("sizeof(float*)  = %zu bytes\n", sizeof(float *));
8     printf("sizeof(double*) = %zu bytes\n", sizeof(double *));
9     printf("sizeof(void*)   = %zu bytes\n", sizeof(void *));
10
11     return 0;
12 }
```

Kết quả cho thấy tất cả các con trỏ đều có cùng kích thước (ví dụ 8 bytes trên hệ 64-bit).

Tuy nhiên, xét thêm đoạn mã sau:

```
1 int a[3] = {10, 20, 30};
2
3 int *p = a;
4 printf("%p\n", p);
5 printf("%p\n", p + 1);
```

Yêu cầu:

1. Tại sao tất cả các con trỏ đều có cùng kích thước nhưng vẫn cần khai báo

kiểu dữ liệu (`int*`, `char*`, ...)?

2. Dự đoán khoảng cách giữa hai địa chỉ được in ra từ `p` và `p + 1`.
3. Nếu thay `int* p` bằng `char* p` (vẫn trỏ tới `a`), điều gì sẽ thay đổi khi in `p + 1`?

Câu hỏi thảo luận:

1. Con trỏ thực sự lưu trữ điều gì trong bộ nhớ?
2. Vì sao trình biên dịch cần biết kiểu dữ liệu mà con trỏ trỏ tới?
3. Phép toán `p + 1` có ý nghĩa gì về mặt địa chỉ bộ nhớ?
4. Liên hệ với mảng: vì sao `a[i]` tương đương với `*(a + i)`?

Bài tập 4: Cấp phát bộ nhớ kiểu C và C++

Mô tả:

Xét chương trình sau:

```
1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;
4
5  int main()
6  {
7      int *p1 = (int *)malloc(sizeof(int));
8      *p1 = 10;
9
10     int *p2 = new int;
11     *p2 = 20;
12
13     cout << *p1 << ' ' << *p2 << endl;
14     cout << p1 << ' ' << p2 << endl;
15
16     delete p1;
17     free(p2);
18
19     cout << *p1 << ' ' << *p2 << endl;
20     cout << p1 << ' ' << p2 << endl;
21 }
```


Yêu cầu:

1. Chương trình trên có biên dịch được không? Có lỗi logic nào không?
2. Hãy chỉ ra vấn đề trong cách giải phóng bộ nhớ của p1 và p2.
3. Dự đoán điều gì có thể xảy ra khi truy cập *p1 và *p2 sau khi đã giải phóng.

Câu hỏi thảo luận:

1. Vì sao malloc cần sizeof(int) trong khi new int thì không?
2. malloc và new khác nhau ở những điểm nào về:
 - kiểu trả về,
 - khởi tạo dữ liệu,
 - cơ chế hoạt động ở mức ngôn ngữ?
3. Vì sao việc sử dụng malloc/free khiến chương trình dễ mắc lỗi hơn so với new/delete?
4. Liên hệ với bài trước: vai trò của sizeof trong việc quản lý bộ nhớ là gì?

Bài tập 5: Quan sát mô hình bộ nhớ: Static Storage, Stack và Heap

Mô tả:

Xét chương trình sau:

```
1 #include <iostream>
2 using namespace std;
3
4 int global_var = 100; // global (data segment)
5
6 int main()
7 {
8     int x = 10;           // stack
9     int y = 20;           // stack
10    int *p = new int(30); // heap
11    int *q = new int(40); // heap
12
13    cout << "Address of x (stack): " << &x << endl;
14    cout << "Address of y (stack): " << &y << endl;
15
16    cout << "Address stored in p (heap object): " << p << endl;
17    cout << "Address stored in q (heap object): " << q << endl;
```

```

18
19     cout << "Address of pointer p (stack): " << &p << endl;
20     cout << "Address of pointer q (stack): " << &q << endl;
21
22     cout << "Address of global_var (data segment): " << &
    global_var << endl;
23
24     delete p;
25     delete q;
26
27     return 0;
28 }

```

Yêu cầu:

1. Chạy chương trình và ghi lại các địa chỉ được in ra.
2. Sắp xếp các địa chỉ theo thứ tự tăng dần và xác định:
 - vùng nhớ nào có địa chỉ thấp hơn,
 - vùng nhớ nào có địa chỉ cao hơn.
3. Xác định các biến thuộc:
 - Static Storage (data segment),
 - Stack,
 - Heap.

Câu hỏi thảo luận:

1. Vì sao p và q (bản thân con trỏ) nằm trên Stack, nhưng giá trị của chúng lại trỏ tới Heap?
2. So sánh địa chỉ của x, y với p, q: chúng có xu hướng tăng hay giảm theo thứ tự khai báo?
3. Địa chỉ của global_var có đặc điểm gì so với Stack và Heap?
4. Có phải lúc nào thứ tự địa chỉ cũng là:
 - Static Storage → Heap → Stack?

Vì sao?

Bài tập 6: Con trỏ như một "cửa sổ" truy cập mảng

Mô tả:

Xét chương trình sau:

```
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4
5 int main()
6 {
7     int n = 5;
8     int *arr = (int *)malloc(n * sizeof(int));
9
10    cout << "Address of pointer arr (stack): " << &arr << endl;
11    cout << "Address stored in arr (heap start): " << arr << endl;
12
13    for (int i = 0; i < n; i++)
14        arr[i] = i * 10;
15
16    cout << "Using arr[i]: " << endl;
17    for (int i = 0; i < n; i++)
18    {
19        cout << "Address of arr[" << i << "] : "
20             << &arr[i] << " value: " << arr[i] << endl;
21    }
22
23    cout << "Using pointer arithmetic: " << endl;
24    for (int i = 0; i < n; i++)
25    {
26        cout << "Address of *(arr + " << i << ") : "
27             << (arr + i) << " value: " << *(arr + i) << endl;
28    }
29
30    free(arr);
31 }
```

Yêu cầu:

1. So sánh kết quả in ra từ hai cách truy cập `arr[i]` và `*(arr + i)`.
2. Nhận xét về khoảng cách giữa các địa chỉ liên tiếp.
3. Giải thích vì sao `arr[i]` tương đương với `*(arr + i)`.

Câu hỏi thảo luận:

1. Con trỏ `arr` thực chất đang biểu diễn điều gì trong bộ nhớ?

2. Phép toán `arr + i` hoạt động như thế nào về mặt địa chỉ?
3. Vì sao các phần tử của mảng nằm liên tiếp trong bộ nhớ?
4. Nếu thay `int` bằng `char`, khoảng cách địa chỉ giữa các phần tử sẽ thay đổi như thế nào?

Bài tập 7: Con trỏ lồng con trỏ và thứ tự giải phóng bộ nhớ

Mô tả:

Xét chương trình sau:

```
1 #include <iostream>
2 #include <cstdlib>
3
4 using namespace std;
5
6 int main()
7 {
8     int **p = (int **)malloc(sizeof(int *));
9     *p = (int *)malloc(sizeof(int));
10    **p = 10;
11
12    cout << p << " " << *p << " " << **p << " " << &p << endl;
13
14    // free(*p);
15    free(p);
16 }
```

Yêu cầu:

1. Giải thích ý nghĩa của từng biến trong chương trình:
 - `p`,
 - `*p`,
 - `**p`.
2. Vẽ sơ đồ bộ nhớ thể hiện mối quan hệ giữa các vùng nhớ trên Stack và Heap.
3. Chương trình trên có bị rò rỉ bộ nhớ (memory leak) hay không? Giải thích.

Câu hỏi thảo luận:

1. Vì sao cần cấp phát hai lần với `malloc`?

2. Điều gì sẽ xảy ra nếu chỉ gọi `free(p)` mà không gọi `free(*p)`?
3. Vì sao cần giải phóng `*p` trước rồi mới giải phóng `p`?
4. Nếu giải phóng `p` trước, điều gì xảy ra với con trỏ `*p`?

III. Bài tập về nhà

Làm các bài tập chủ đề **Quản lý Bộ nhớ** (*[Memory Management]*) trên trang Coding. Liên hệ với giảng viên thực hành về thời hạn làm bài.

Nhắc lại: Để biên dịch và thực thi bài làm bằng C++, trong mã nguồn làm bài sẽ cần để ở dòng đầu tiên đoạn mã `// .cpp`, ví dụ như:

```
1 // .cpp
2 #include <iostream>
3 using namespace std;
4 ...
```